DOCUMENT // INDUSTRIAL FIELD MANUAL

BACKEND // JAVA

ARCHIVE RECORD // TECHNICAL NOTE

MANUAL DE INGENIERÍA · DIVISIÓN DE SISTEMAS

SERIE: JAVA-01 · ÁREA: BACKEND · ESTADO: OPERATIVO

DATA BLOCK // SYSTEM OUTPUT

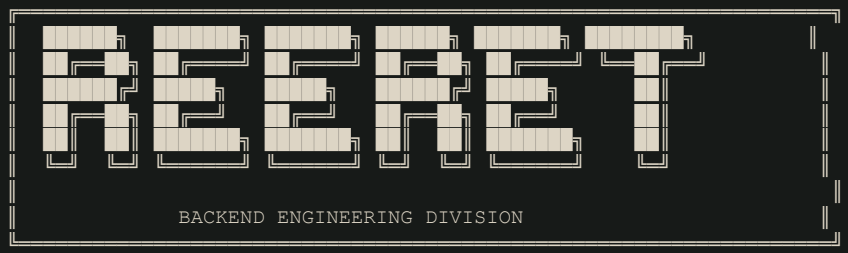


TABLA DE OPERACIONES

- 01 · 🌐 Servidores web
- 02 · 📡 Protocolo HTTP
- 03 · 🔀 Procesos y Threads
- 04 · ⚙️ Arquitectura MVC
- 05 · 🧱 CRUD & POJO
- 06 · 🌱 Spring
- 07 · 🚀 Spring Boot
- 08 · 🧠 Patrones de diseño
- 09 · 🐘 PHP

DOCUMENT // INDUSTRIAL FIELD MANUAL

01 // 🌐 SERVIDORES WEB

ARCHIVE RECORD // TECHNICAL NOTE

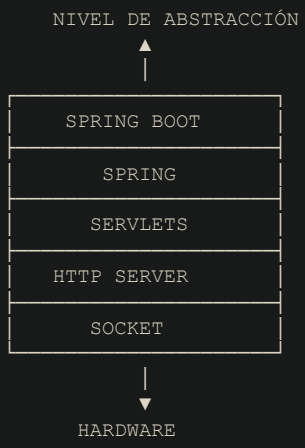
REGISTRO DE INGENIERÍA

Antes de emplear maquinaria de alto nivel, conviene comprender qué ocurre en las entrañas del sistema.

Existen diferentes niveles para construir un servidor web utilizando Java.

Podemos recorrer el sistema desde sus componentes más básicos hasta una infraestructura completa:

DATA BLOCK // SYSTEM OUTPUT

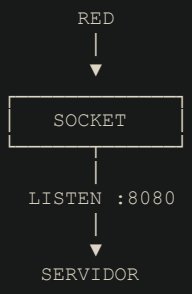


🔩 01.1 · SOCKET

Para una primera **prueba de concepto**, podemos trabajar directamente con un `Socket` de Java.

La idea fundamental:

DATA BLOCK // SYSTEM OUTPUT



El servidor puede quedar escuchando en un puerto determinado:

DATA BLOCK // SYSTEM OUTPUT

```
localhost:8080
```

Esto permite comprender qué ocurre cuando una máquina recibe una conexión.

ARCHIVE RECORD // TECHNICAL NOTE

NOTA DE TALLER

Trabajar directamente con sockets es una buena forma de comprender la maquinaria que posteriormente ocultan los frameworks.

01.2 · SERVIDOR HTTP BÁSICO

Java permite levantar un servidor HTTP básico.

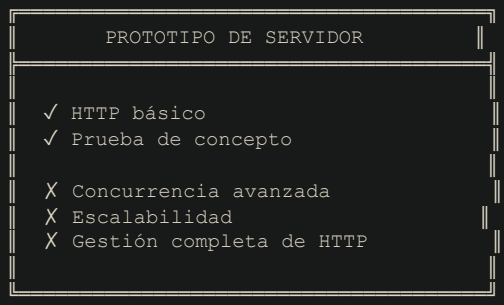
Es suficiente para construir una **Proof of Concept (PoC)** y estudiar el funcionamiento mínimo de un servidor web.

Sin embargo, cuando aumenta la carga aparecen problemas relacionados con:

- Concurrencia.
- Gestión de conexiones.
- Peticiones HTTP.

- Escalabilidad.
- Gestión de recursos.

DATA BLOCK // SYSTEM OUTPUT



Es, por tanto, una máquina de demostración, no necesariamente la infraestructura que utilizaríamos para producción.

DOCUMENT // INDUSTRIAL FIELD MANUAL

02 // 🔧 SERVLETS

El siguiente nivel de la maquinaria son los **Servlets**.

Los Servlets proporcionan una abstracción superior para trabajar con HTTP y permiten delegar parte de la infraestructura en un **contenedor**.

El programador ya no necesita encargarse directamente de todos los engranajes internos.

DATA BLOCK // SYSTEM OUTPUT



El contenedor se encarga de numerosas tareas que el desarrollador no debería tener que implementar desde cero.

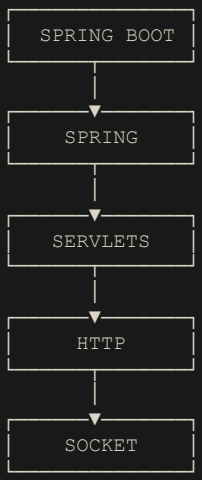
DOCUMENT // INDUSTRIAL FIELD MANUAL

03 // FRAMEWORKS

Sobre esta infraestructura aparecen los frameworks.

En nuestro caso:

DATA BLOCK // SYSTEM OUTPUT



ARCHIVE RECORD // TECHNICAL NOTE

PRINCIPIO DE INGENIERÍA

Un framework permite reutilizar infraestructura, patrones y componentes existentes en lugar de construir toda la maquinaria desde cero.

DOCUMENT // INDUSTRIAL FIELD MANUAL

04 // HTTP

Toda operación web comienza con una comunicación entre dos piezas fundamentales:

DATA BLOCK // SYSTEM OUTPUT



El **cliente solicita**.

El **servidor procesa**.

El **servidor responde**.

⚒️ MÉTODOS HTTP

MÉTODO	OPERACIÓN
`GET`	Obtener información
`POST`	Crear o enviar información
`PUT`	Reemplazar un recurso
`PATCH`	Modificar parcialmente un recurso
`HEAD`	Obtener únicamente las cabeceras
`DELETE`	Eliminar un recurso

Una petición HTTP puede contener:

```
DATA BLOCK // SYSTEM OUTPUT

┌──────────────────────────────┐
│ HTTP REQUEST                 │
├──────────────────────────────┤
│ METHOD                       │
│ URL                          │
│ HEADERS                      │
│ QUERY PARAMETERS             │
│ BODY                         │
└──────────────────────────────┘
```

🔍 QUERY PARAMETERS

Los parámetros de consulta forman parte de la URL:

```
DATA BLOCK // SYSTEM OUTPUT

/users?id=42
```

Aquí:

DATA BLOCK // SYSTEM OUTPUT

```
id = 42
```

es un **query parameter**.

DOCUMENT // INDUSTRIAL FIELD MANUAL

05 // 🧵 PROCESOS & THREADS

ARCHIVE RECORD // TECHNICAL NOTE

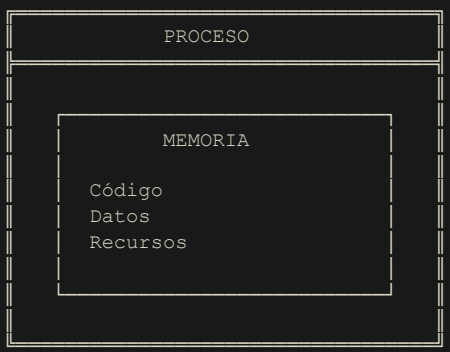
REGISTRO DE MAQUINARIA

Para comprender la concurrencia debemos distinguir entre procesos y threads.

⚙️ PROCESO

Un proceso dispone de su propio espacio de memoria.

DATA BLOCK // SYSTEM OUTPUT



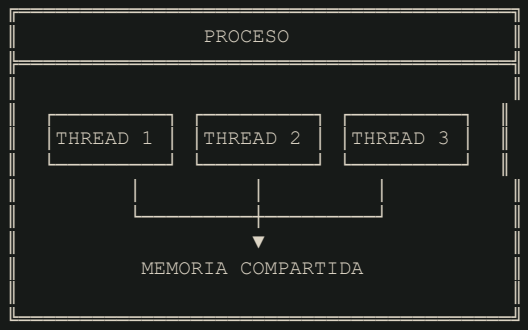
En condiciones normales, los procesos están aislados entre sí.

⚙️ THREAD

Un proceso puede contener varios threads.

Estos threads comparten el espacio de memoria del proceso:

DATA BLOCK // SYSTEM OUTPUT



Los threads son más ligeros que procesos independientes, pero compartir memoria introduce nuevos problemas.

DOCUMENT // INDUSTRIAL FIELD MANUAL

06 // 🔄 CONTEXT SWITCH

El sistema operativo debe alternar entre los diferentes procesos y threads que se están ejecutando.

Este mecanismo recibe el nombre de:

ARCHIVE RECORD // TECHNICAL NOTE

Context Switch · Cambio de contexto

Los threads permiten una ejecución concurrente con menor coste que crear procesos independientes, aunque el sistema sigue teniendo que gestionar los cambios de ejecución.

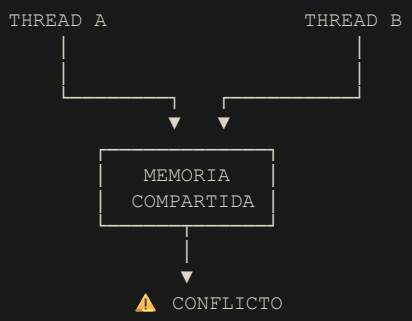
⚠️ PELIGRO: MEMORIA COMPARTIDA

Compartir memoria significa que varios threads pueden acceder simultáneamente al mismo recurso.

Esto puede provocar:

- Condiciones de carrera.
- Datos inconsistentes.
- Accesos simultáneos.
- Problemas de sincronización.

DATA BLOCK // SYSTEM OUTPUT



🛡️ THREAD-SAFE

Un componente **thread-safe** está diseñado para comportarse correctamente cuando es utilizado simultáneamente por varios threads.

Java proporciona diferentes mecanismos de sincronización y concurrencia para controlar estos accesos.

DOCUMENT // INDUSTRIAL FIELD MANUAL

07 // MVC

ARCHIVE RECORD // TECHNICAL NOTE

MODELO ARQUITECTÓNICO

Separar responsabilidades evita convertir la aplicación en una única máquina imposible de mantener.

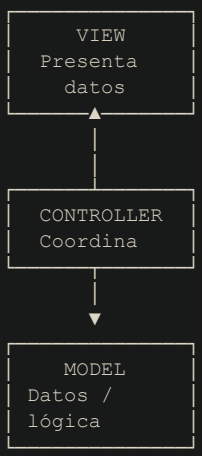
MVC significa:

DATA BLOCK // SYSTEM OUTPUT

```
MODEL
VIEW
CONTROLLER
```

ESTRUCTURA

DATA BLOCK // SYSTEM OUTPUT



La idea principal es **separar responsabilidades**.

📦 MODEL

El modelo representa los datos que maneja la aplicación y la lógica asociada a ellos.

Por ejemplo:

DATA BLOCK // SYSTEM OUTPUT

```
public class User {

    private String name;
    private String email;

}
```

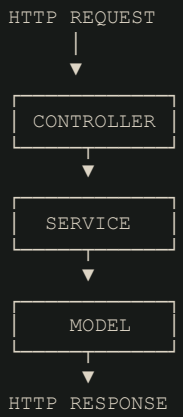
En arquitecturas más completas pueden existir otras capas entre el controlador y los datos.

🎛️ CONTROLLER

El controlador recibe las peticiones y coordina las diferentes partes de la aplicación.

Un flujo habitual:

DATA BLOCK // SYSTEM OUTPUT



DOCUMENT // INDUSTRIAL FIELD MANUAL

08 // CRUD

Las cuatro operaciones fundamentales sobre datos:

DATA BLOCK // SYSTEM OUTPUT

```
╔═══════════════════════════════════╗
║              CRUD                 ║
╠═══════════════════════════════════╣
║                                   ║
║  C · CREATE   → Crear             ║
║  R · READ     → Leer              ║
║  U · UPDATE   → Actualizar        ║
║  D · DELETE   → Eliminar          ║
║                                   ║
╚═══════════════════════════════════╝
```

En una API:

DATA BLOCK // SYSTEM OUTPUT

```
  POST    /users       → CREATE
  GET     /users/1     → READ
  PUT     /users/1     → UPDATE
  DELETE  /users/1     → DELETE
```

DOCUMENT // INDUSTRIAL FIELD MANUAL

09 // POJO

POJO

ARCHIVE RECORD // TECHNICAL NOTE

Plain Old Java Object

Es un objeto Java sencillo utilizado habitualmente para representar o transportar datos.

DATA BLOCK // SYSTEM OUTPUT

```
public class User {

    private String name;
    private String email;

    // getters / setters
}
```

No necesita una infraestructura especialmente compleja para existir: es simplemente una clase Java.

DOCUMENT // INDUSTRIAL FIELD MANUAL

10 // SPRING

ARCHIVE RECORD // TECHNICAL NOTE

SPRING ENGINEERING DIVISION

Spring es un framework para desarrollar aplicaciones Java.

Su propósito es proporcionar una infraestructura reutilizable y abstraer numerosas tareas que, de otro modo, tendríamos que implementar manualmente.

DATA BLOCK // SYSTEM OUTPUT



DOCUMENT // INDUSTRIAL FIELD MANUAL

11 // SPRING BOOT

Spring Boot simplifica la configuración y puesta en marcha de aplicaciones Spring.

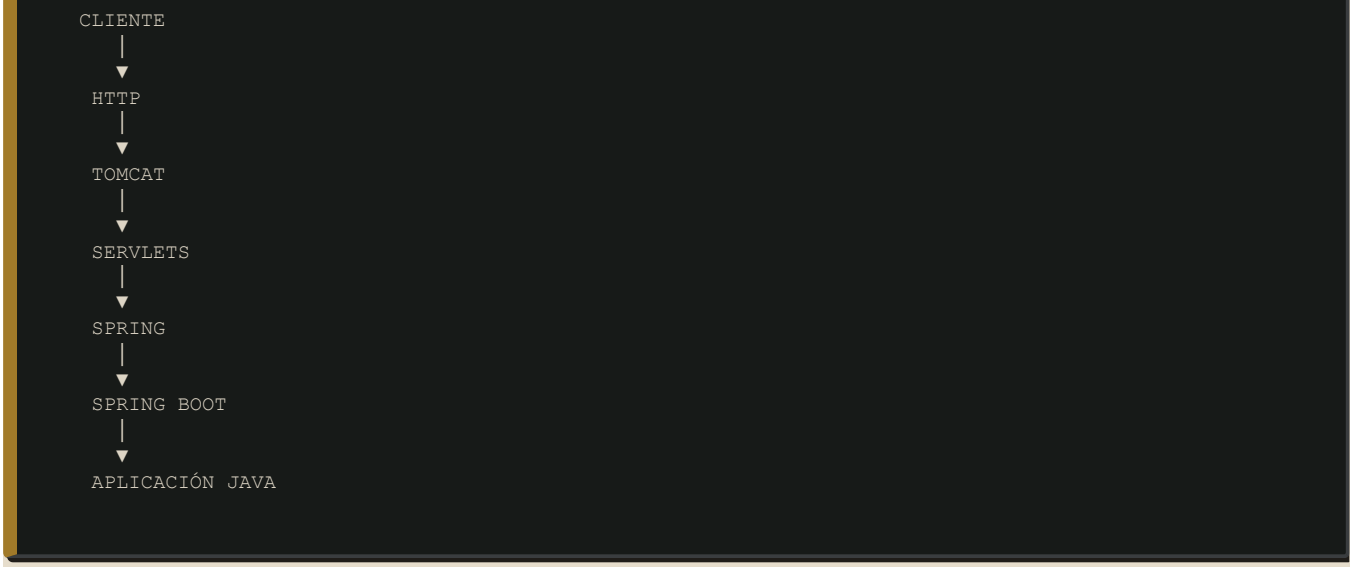
Una de sus características habituales es utilizar un servidor embebido, como **Tomcat**.

DATA BLOCK // SYSTEM OUTPUT



La cadena de funcionamiento queda aproximadamente así:

DATA BLOCK // SYSTEM OUTPUT



DOCUMENT // INDUSTRIAL FIELD MANUAL

12 // 🧠 IOC & DI

Spring se apoya en conceptos fundamentales como:

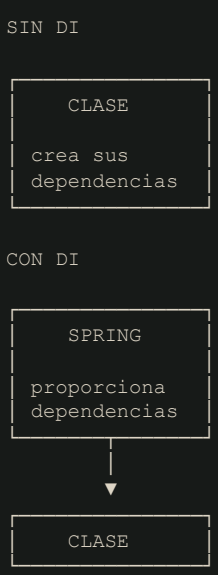
IOC · INVERSION OF CONTROL

La responsabilidad de crear y gestionar determinados objetos se delega al framework.

DI · DEPENDENCY INJECTION

Las dependencias necesarias para una clase son proporcionadas desde el exterior.

DATA BLOCK // SYSTEM OUTPUT



DOCUMENT // INDUSTRIAL FIELD MANUAL

13 // ⚒ PATRONES DE DISEÑO

Los **patrones de diseño** son soluciones reutilizables para problemas habituales del desarrollo de software.

📚 REFERENCIA

ARCHIVE RECORD // TECHNICAL NOTE

Refactoring.Guru
Catálogo de patrones de diseño.

https://refactoring.guru/design-patterns

En Java pueden implementarse mediante diferentes mecanismos del lenguaje, como:

- `interface`
- Composición.

- Herencia.
- Polimorfismo.
- Encapsulación.

DOCUMENT // INDUSTRIAL FIELD MANUAL

14 // 🐘 PHP

ARCHIVE RECORD // TECHNICAL NOTE

ÁREA RESERVADA

La siguiente fase del manual estará dedicada al backend con **PHP**.

DATA BLOCK // SYSTEM OUTPUT

```
NEXT ENGINE: PHP
STATUS: PENDING
```

DOCUMENT // INDUSTRIAL FIELD MANUAL

⚙ REGISTRO DE MANTENIMIENTO

DATA BLOCK // SYSTEM OUTPUT

```
┌──────────────────────────────────────────┐
│ ESTADO DEL MANUAL                        │
├──────────────────────────────────────────┤
│                                          │
│ [✓] Socket                               │
│ [✓] HTTP                                 │
│ [✓] Servlets                             │
│ [✓] Procesos y Threads                   │
│ [✓] MVC                                  │
│ [✓] CRUD                                 │
│ [✓] POJO                                 │
│ [✓] Spring                               │
│ [✓] Spring Boot                          │
│ [✓] IoC / DI                             │
│ [✓] Patrones de diseño                   │
│ [ ] PHP                                  │
│                                          │
└──────────────────────────────────────────┘
```

ARCHIVE RECORD // TECHNICAL NOTE

FIN DEL REGISTRO

JAVA BACKEND ENGINEERING // DOCUMENT 01

Mantener la maquinaria limpia. Comprender cada engranaje.